

Multi-Agent Collaboration for Mathematical Reasoning: Accuracy Gains and Error Reduction on GSM8K

Diego Alberto Ramírez Madriz
Universitat Politècnica de Catalunya (UPC)
`diego.alberto.ramirez@estudiantat.upc.edu`

Supervisor: Sergi Bermejo, PhD
Universitat Politècnica de Catalunya (UPC)

December 2025

1 Introduction

Large Language Models (LLMs) can solve many mathematical word problems but still show recurring numerical and logical mistakes when used as single agents. Recent work on multi-agent collaboration [1, 2] suggests that assigning different roles, such as proposer, critic, and refiner, or using proposer + judge architectures, can help reduce these errors by improving the intermediate reasoning process.

This project examines whether such structured multi-agent collaboration leads to higher accuracy on GSM8K problem set compared to a single agent baseline, and what additional computational cost these improvements require. The study focuses on two questions:

- Does collaboration between multiple specialized agents increase accuracy on GSM8K against a single agent?
- How do these gains relate to the extra compute used at inference time?

The central aim is to assess the error reduction benefits (or accuracy gains) of different agentic architectures, with computational cost noted as a practical consequence rather than a primary object of study.

2 Methodology

2.1 Model, Task and Dataset

Using the qwen2.5 7B model, GSM8K is evaluated on, a benchmark of grade-school mathematical problems requiring multi-step reasoning. For each token budget configuration, to ensure comparability across architectures a fixed subset of $N = 200$ items is tested.

2.2 Architectures

We compare the following architectures:

Single. A standard single agent baseline using chain-of-thought prompting.

MAV (Proposer-Critic-Synthesizer). Our MAV implementation follows the structure introduced in [1].

1. **Proposer:** generates a detailed solution for the problem.
2. **Critic:** examines the reasoning, identifying possible errors or inconsistencies.
3. **Refiner:** integrates the Proposer’s reasoning and the Critic’s analysis into a final answer.

Proposer + Judge (M = 3, 5; loose / strict). Multiple Proposers (M) independently generate candidate solutions. A Judge selects an answer between the M solutions. There are two different token budget styles.

- *Loose:* the defined token budget is available individually to each agent participating in the problem.
- *Strict:* the defined token budget is available for the whole architecture, in other words, the agents share a token budget.

All architectures use the same underlying LLM and closely aligned prompts to maintain fairness.

2.3 Experimental Setup

Experiments were run locally on an M1 Pro MacBook using a quantized open-source instruction-tuned qwen2.5-7B-instruct-q_4k loaded via llama-cpp. We sweep three token budgets:

$$\text{max_tokens} \in \{256, 750, 1536\},$$

and evaluate each architecture on the same 200 problems.

For each configuration we record:

- **Accuracy**
- **Error rate** = $1 - \text{accuracy}$
- **Tokens per correct answer**
- **Seconds per correct answer**

A 95% confidence interval for accuracy is estimated with a binomial approximation:

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{N}}.$$

Error reduction relative to the single-agent baseline is:

$$\text{ErrorRed} = \frac{\text{err}_{\text{single}} - \text{err}_{\text{mode}}}{\text{err}_{\text{single}}} \times 100\%,$$

with all metrics computed from aggregated CSV logs (obtained in the experimentation phase) using a dedicated analysis module.

mode	total_budget	accuracy	ci95_low	ci95_high	error_rate	error_reduction_pct	tokens_per_correct	secs_per_correct
mav	256	0.900	0.858	0.942	0.100	58.333	423.794	21.879
prop_judge_M3_loose	256	0.865	0.818	0.912	0.135	43.750	329.543	16.317
prop_judge_M5_loose	256	0.900	0.858	0.942	0.100	58.333	518.678	24.809
single	256	0.760	0.701	0.819	0.240	0.000	118.007	5.287
mav	750	0.910	0.870	0.950	0.090	57.143	476.813	24.484
prop_judge_M3_loose	750	0.860	0.812	0.908	0.140	33.333	331.047	16.567
prop_judge_M5_loose	750	0.875	0.829	0.921	0.125	40.476	535.177	26.854
single	750	0.790	0.734	0.846	0.210	0.000	114.981	5.098
mav	1536	0.920	0.882	0.958	0.080	60.976	460.478	23.938
prop_judge_M3_loose	1536	0.840	0.789	0.891	0.160	21.951	341.458	16.918
prop_judge_M5_loose	1536	0.850	0.801	0.899	0.150	26.829	552.976	26.520
single	1536	0.795	0.739	0.851	0.205	0.000	115.597	5.363

Table 1: Main experimental results across architectures and token budgets.

3 Results

3.1 Accuracy and Error Reduction

Table 1 summarizes the results for the single-agent baseline, MAV, and the loose judge configurations (M3, M5). Strict judge variants are omitted for clarity, as some, especially the 256 budget variant, under perform the baseline because of its strong token restriction. This outlines that such types of architectures have a higher need of computational cost flexibility.

Across all budgets, MAV consistently achieves higher accuracy. At 256 tokens, the single-agent accuracy is 0.760 (error 0.240), whereas MAV reaches 0.900 (error 0.100), obtaining an error reduction of about 58%. At 750 and 1536 tokens, MAV maintains accuracy between 0.910 and 0.920, corresponding to error reductions between 57% and 61%.

Loose judge configurations also improve over the baseline, typically achieving error reductions between 22% and 44%, though remaining below MAV.

Figure 1 shows accuracy as a function of token budget with 95% confidence intervals. MAV and the best loose judge variants lie clearly above the single-agent baseline, indicating that the improvements are unlikely to be due to sampling variability.

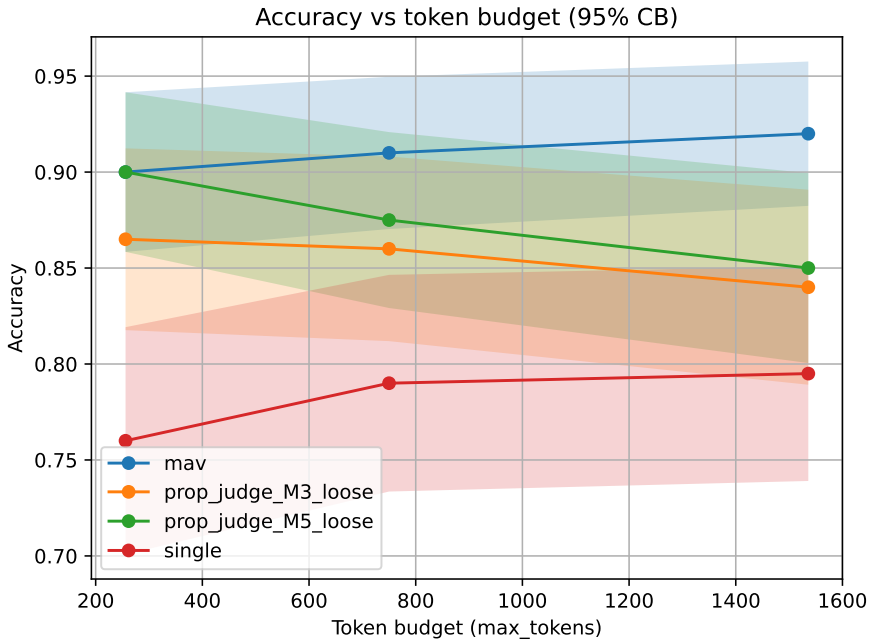


Figure 1: Accuracy vs token budget for selected architectures. Shaded regions indicate 95% confidence intervals.

3.2 Cost–Accuracy Trade-Off

Multi-agent systems require substantially more computation. Relative to the single-agent baseline:

- MAV uses approximately $3.6\text{--}4.1\times$ more tokens per correct answer and $4.1\text{--}4.8\times$ more time per correct answer.
- Loose judge configurations are somewhat cheaper ($2.8\text{--}3.0\times$ token cost) but offer smaller accuracy gains.

Figure 2 illustrates the trade-off between token cost and error reduction. MAV occupies the “high cost, high error reduction” region, while loose judge systems sit in a middle region of moderate cost and moderate improvement. The single-agent baseline is the most efficient but also the least accurate.

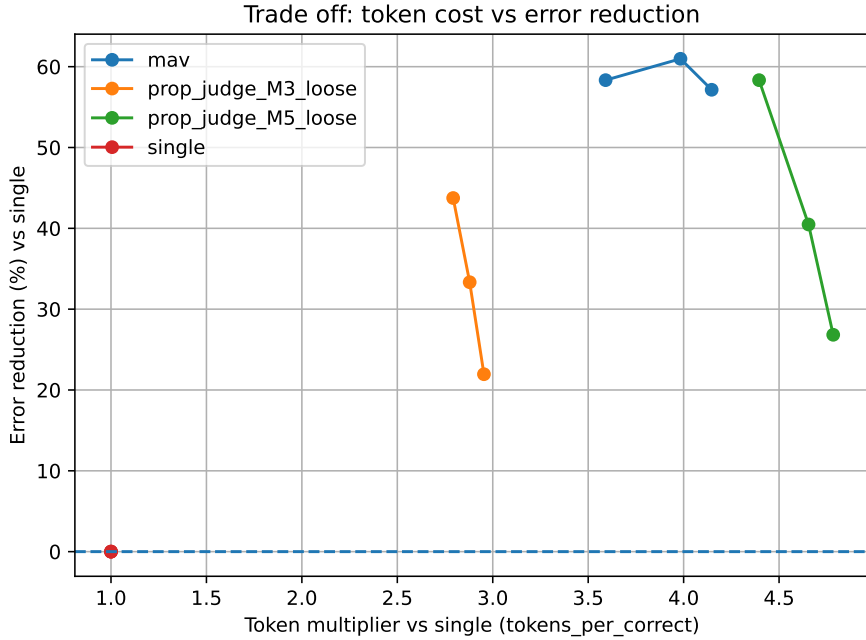


Figure 2: Trade-off between token multiplier vs single (x-axis) and error reduction relative to single (y-axis).

These findings show the standard pattern observed in earlier multi-agent work: higher accuracy is obtained at the expense of more test-time compute.

4 Discussion and Conclusions

The results highlight three main observations:

- **Accuracy and reliability.** In this experiment, MAV improves accuracy by 10–18 percentage points over the single-agent system, reducing error by roughly 55–61%. Loose judge configurations produce smaller but consistent gains. These multi-agent setups remain comparatively stable across different token budgets. This results align with observations in prior multi-agent work [1].
- **Cost.** These improvements require substantially more computation: MAV needs about $3.6\text{--}4.1\times$ the tokens and $4.1\text{--}4.8\times$ the time per correct solution. Loose judge variants lie

between MAV and the baseline. Multi-agent methods are therefore most appealing when correctness is prioritized over efficiency.

- **Number of agents in architectures.** The M5 loose Judge configuration showed that increasing the number of proposers past a certain point does not necessarily yield better accuracy. Its performance was comparable to M3 loose, despite using substantially more computational resources.

In short, the experiments show that multi-agent collaboration can turn additional test time compute into significantly fewer reasoning errors on GSM8K throughout different architectures. For applications where accuracy and reliability matter more than minimal latency or token usage, these types of multi agent architectures offer a clear advantage over single agent approaches.

5 Limitations and Future Work

The evaluation was limited to a 200-item subset of GSM8K and relied on a single model with manually designed agent prompts. These choices simplify experimentation but restrict generality.

Future work includes evaluating the full dataset, trying alternative models, and developing more principled or learned judge and critic strategies.

References

- [1] Shalev Lifshitz, Sheila A. McIlraith, and Yilun Du. Multi-agent verification: Scaling test-time compute with multiple verifiers. *arXiv preprint arXiv:2502.20379*, 2025.
- [2] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D. Nguyen. Multi-agent systems: Multi-agent collaboration mechanisms: A survey of llms. *arXiv preprint arXiv:2501.06322*, 2025.